WEEK 10: ADVANCED SQL QUERY SYSTEM SCENARIO

-S.KAVIYA(E25AI119)

Scenario:
The ArtWork Shop wants to perform advanced analysis on customer purchases,product pricing,and sales performance.The management team needs complex reports to identify high-value customers,expensive products, and business trends. It will implement subqueries and nested queries to extract deeper insights from the Artwork Shop Database.

Tasks to be Done:
1. Implementation of Subqueries and Nested Queries

Subqueries and Nested Queries are SQL techniques used to retrieve information by executing one query within another query. They help perform advanced database operations by using the output of an inner query to obtain the required results in the outer query.
Types Covered:

- Single-Row Subquery : Returns one row.
- Multi-Row Subquery : Returns more than one row.
- Nested Subquery : A query written inside another query.

2. Find Products Above Average Price Task:

1. Find products priced above the average price.

```
SQL> SELECT AVG(PRICE) FROM PRODUCT;

AVG(PRICE)
----------
17694.6667

SQL> SELECT PRODUCT_NAME,PRICE FROM PRODUCT
  2  WHERE PRICE>(SELECT AVG(PRICE) FROM PRODUCT);

PRODUCT_NAME
--------------------------------------------------------------------
     PRICE
----------
Laptops
     67500

Croptops and shirts
     45000

Television
     75000


PRODUCT_NAME
--------------------------------------------------------------------
     PRICE
----------
Tiles
     55000
```

2.Display product name, category, and price.

```
SQL> SELECT PRODUCT_NAME,CATEGORY_ID,PRICE
  2  FROM PRODUCT;

PRODUCT_NAME
--------------------------------------------------------------------------------
CATEGOR      PRICE
------- ----------
Laptops
SEM0001      67500

Croptops and shirts
SEM0002      45000

Fish tank
SEM0003       3400


PRODUCT_NAME
--------------------------------------------------------------------------------
CATEGOR      PRICE
------- ----------
Television
SEM0004      75000

Unruled notes
SEM0005       7500

Tiles
SEM0006      55000


PRODUCT_NAME
--------------------------------------------------------------------------------
CATEGOR      PRICE
------- ----------
Headphones
SEM0001       2500

Keyboard
SEM0001       1500

T-Shirts
SEM0002        800


PRODUCT_NAME
--------------------------------------------------------------------------------
CATEGOR      PRICE
------- ----------
Jeans
SEM0002       1800

Notebooks
SEM0003        100

Pens
SEM0003         20


PRODUCT_NAME
--------------------------------------------------------------------------------
CATEGOR      PRICE
------- ----------
Mixer Grinder
SEM0004       3500

Mobile Charger
SEM0005        600

Rice Bag
SEM0006       1200


15 rows selected.
```

3.Identify expensive products in each category.

```
SQL> SELECT P1.PRODUCT_NAME, P1.CATEGORY_ID, P1.PRICE
  2  FROM PRODUCT P1
  3  WHERE P1.PRICE > (SELECT AVG(P2.PRICE)
  4  FROM PRODUCT P2
  5  WHERE P1.CATEGORY_ID = P2.CATEGORY_ID);

PRODUCT_NAME
--------------------------------------------------------------------------------
CATEGOR      PRICE
------- ----------
Laptops
SEM0001      67500

Croptops and shirts
SEM0002      45000

Fish tank
SEM0003       3400


PRODUCT_NAME
--------------------------------------------------------------------------------
CATEGOR      PRICE
------- ----------
Television
SEM0004      75000

Unruled notes
SEM0005       7500

Tiles
SEM0006      55000


6 rows selected.
```

4. Find products with price higher than a selected category average.

```
SQL> SELECT PRODUCT_NAME, CATEGORY_ID, PRICE
  2  FROM PRODUCT
  3  WHERE CATEGORY_ID = 'SEM0001'
  4  AND PRICE > (SELECT AVG(PRICE)
  5  FROM PRODUCT
  6  WHERE CATEGORY_ID = 'SEM0001');

PRODUCT_NAME
--------------------------------------------------------------------------------
CATEGOR      PRICE
------- ----------
Laptops
SEM0001      67500
```

3. Identify Customers with Maximum Purchases Task:

1. Customer who spent the highest amount.

```
SQL> SELECT CUSTOMER_ID, MAX(TOTAL_AMOUNT)
  2  FROM ORDERS
  3  GROUP BY CUSTOMER_ID;

CUSTOMER_ID          MAX(TOTAL_AMOUNT)
-------------------- -----------------
C-1001                           70000
C-1004                           75000
C-1002                           67500
C-1003                           45000
C-1005                           60000

SQL> SELECT MAX(TOTAL_AMOUNT)
  2  FROM ORDERS;

MAX(TOTAL_AMOUNT)
-----------------
            75000
```

```
SQL> SELECT CUSTOMER_ID, TOTAL_AMOUNT
  2  FROM ORDERS
  3  WHERE TOTAL_AMOUNT = (SELECT MAX(TOTAL_AMOUNT) FROM ORDERS);

CUSTOMER_ID          TOTAL_AMOUNT
-------------------- ------------
C-1004                      75000
```

2. Customers with maximum number of orders.

```
SQL> SELECT CUSTOMER_ID, COUNT(*) AS NO_OF_ORDERS
  2  FROM ORDERS
  3  GROUP BY CUSTOMER_ID
  4  HAVING COUNT(*) = (
  5      SELECT MAX(NO_OF_ORDERS)
  6      FROM (
  7          SELECT COUNT(*) AS NO_OF_ORDERS
  8          FROM ORDERS
  9          GROUP BY CUSTOMER_ID));

CUSTOMER_ID          NO_OF_ORDERS
-------------------- ------------
C-1001                          2
C-1004                          2
C-1002                          2
C-1003                          2
C-1005                          2
```

3. Customers whose spending is above average.

```
SQL> SELECT AVG(TOTAL_AMOUNT) FROM ORDERS GROUP BY CUSTOMER_ID;

AVG(TOTAL_AMOUNT)
-----------------
            62500
            39200
            56250
            24200
            33750

SQL> SELECT CUSTOMER_ID, SUM(TOTAL_AMOUNT) AS TOTAL_SPENDING
  2  FROM ORDERS
  3  GROUP BY CUSTOMER_ID
  4  HAVING SUM(TOTAL_AMOUNT) > (
  5      SELECT AVG(TOTAL_SPENDING)
  6      FROM (
  7          SELECT SUM(TOTAL_AMOUNT) AS TOTAL_SPENDING
  8          FROM ORDERS
  9          GROUP BY CUSTOMER_ID
 10      )
 11  );

CUSTOMER_ID          TOTAL_SPENDING
-------------------- --------------
C-1001                       125000
C-1002                       112500
```

4. Top 5 valuable customers.

```
SQL> SELECT * FROM(SELECT CUSTOMER_ID, SUM(TOTAL_AMOUNT) AS TOTAL_SPENDING
FROM ORDERS
  2   GROUP BY CUSTOMER_ID
  3   ORDER BY TOTAL_SPENDING DESC)
  4   WHERE ROWNUM <= 5;

CUSTOMER_ID          TOTAL_SPENDING
-------------------- --------------
C-1001                       125000
C-1002                       112500
C-1004                        78400
C-1005                        67500
C-1003                        48400
```

4. Generate Complex Business Queries

Query 1: Best-Selling Product

Find products with maximum sales quantity.

Tasks:

● Display product name.
● Total quantity sold.
● Total revenue generated.

```
SQL> SELECT P.PRODUCT_NAME,
  2         SUM(OD.QUANTITY) AS TOTAL_QUANTITY_SOLD,
  3         SUM(OD.QUANTITY*OD.PRICE) AS TOTAL_REVENUE
  4  FROM PRODUCT P, ORDER_DETAILS OD
  5  WHERE P.PRODUCT_ID = OD.PRODUCT_ID
  6  GROUP BY P.PRODUCT_ID, P.PRODUCT_NAME
  7  HAVING SUM(OD.QUANTITY) = (SELECT MAX(SUM(QUANTITY))
  8  FROM ORDER_DETAILS
  9  GROUP BY PRODUCT_ID);

PRODUCT_NAME
--------------------------------------------------------------------
TOTAL_QUANTITY_SOLD TOTAL_REVENUE
------------------- -------------
Laptops
                  2        135000

Croptops and shirts
                  2         90000

Fish tank
                  2          6800
```

Query 2: High-Value Customers
Find customers whose total purchase amount is greater than average customer spending

```
SQL> SELECT CUSTOMER_ID, SUM(TOTAL_AMOUNT) AS TOTAL_SPENT
  2  FROM ORDERS
  3  GROUP BY CUSTOMER_ID
  4  HAVING SUM(TOTAL_AMOUNT) > (SELECT AVG(SUM(TOTAL_AMOUNT))
  5  FROM ORDERS
  6  GROUP BY CUSTOMER_ID);

CUSTOMER_ID          TOTAL_SPENT
-------------------- -----------
C-1001                    125000
C-1002                    112500
```

Query 3: Category Performance Analysis
Find:
● Highest revenue-generating category.
● Number of products sold in each category.
● Average product price by category.

```
SQL> SELECT C.CATEGORY_ID,
  2         SUM(OD.QUANTITY*OD.PRICE) AS TOTAL_REVENUE,
  3         SUM(OD.QUANTITY) AS TOTAL_PRODUCTS_SOLD,
  4         AVG(P.PRICE) AS AVERAGE_PRODUCT_PRICE
  5  FROM CATEGORY C, PRODUCT P, ORDER_DETAILS OD
  6  WHERE C.CATEGORY_ID = P.CATEGORY_ID
  7  AND P.PRODUCT_ID = OD.PRODUCT_ID
  8  GROUP BY C.CATEGORY_ID
  9  ORDER BY TOTAL_REVENUE DESC;

CATEGOR TOTAL_REVENUE TOTAL_PRODUCTS_SOLD AVERAGE_PRODUCT_PRICE
------- ------------- ------------------- --------------------
SEM0001        135000                   2                67500
SEM0002         90000                   2                45000
SEM0004         75000                   1                75000
SEM0006         55000                   1                55000
SEM0005          7500                   1                 7500
SEM0003          6800                   2                 3400

6 rows selected.
```

Query 4: Customer Purchase History

Display:

- Customer name
- Number of orders
- Total purchase amount
- Most purchased product

```
SQL> SELECT C.FIRST_NAME, COUNT(O.ORDER_ID), SUM(O.TOTAL_AMOUNT),
  2  (SELECT P.PRODUCT_NAME FROM PRODUCT P, ORDER_DETAILS OD
  3  WHERE P.PRODUCT_ID=OD.PRODUCT_ID
  4  AND OD.ORDER_ID=O.ORDER_ID AND ROWNUM=1)
  5  FROM CUSTOMER C, ORDERS O
  6  WHERE C.CUSTOMER_ID=O.CUSTOMER_ID
  7  GROUP BY C.CUSTOMER_ID, C.FIRST_NAME, O.ORDER_ID;

FIRST_NAME           COUNT(O.ORDER_ID) SUM(O.TOTAL_AMOUNT)
-------------------- ----------------- -------------------
(SELECTP.PRODUCT_NAMEFROMPRODUCTP,ORDER_DETAILSODWHEREP.PRODUCT_ID=OD.PRODUCT_ID
--------------------------------------------------------------------------------
Beema                                1               55000
Tiles

priya                                1               67500
Laptops

Reena                                1                7500
Unruled notes


FIRST_NAME           COUNT(O.ORDER_ID) SUM(O.TOTAL_AMOUNT)
-------------------- ----------------- -------------------
(SELECTP.PRODUCT_NAMEFROMPRODUCTP,ORDER_DETAILSODWHEREP.PRODUCT_ID=OD.PRODUCT_ID
--------------------------------------------------------------------------------
Anu                                  1               75000
Television

Reena                                1               60000


Beema                                1               70000
Laptops


FIRST_NAME           COUNT(O.ORDER_ID) SUM(O.TOTAL_AMOUNT)
-------------------- ----------------- -------------------
(SELECTP.PRODUCT_NAMEFROMPRODUCTP,ORDER_DETAILSODWHEREP.PRODUCT_ID=OD.PRODUCT_ID
--------------------------------------------------------------------------------
Anu                                  1                3400
Fish tank

priya                                1               45000
Croptops and shirts

Arun                                 1               45000
Croptops and shirts


FIRST_NAME           COUNT(O.ORDER_ID) SUM(O.TOTAL_AMOUNT)
-------------------- ----------------- -------------------
(SELECTP.PRODUCT_NAMEFROMPRODUCTP,ORDER_DETAILSODWHEREP.PRODUCT_ID=OD.PRODUCT_ID
--------------------------------------------------------------------------------
Arun                                 1                3400
Fish tank


10 rows selected.
```

5. Prepare Advanced SQL Report:

Report 1: Premium Product Report

Display:

- Product Name
- Category
- Price

● Stock Availability Condition:
Products above average price.

```
SQL> SELECT P.PRODUCT_NAME, C.DESCRIPTION, P.PRICE, P.STOCK_QUANTITY
  2  FROM PRODUCT P, CATEGORY C
  3  WHERE P.CATEGORY_ID=C.CATEGORY_ID
  4  AND P.PRICE > (SELECT AVG(PRICE) FROM PRODUCT);

PRODUCT_NAME
--------------------------------------------------------------------------------
DESCRIPTION                                        PRICE STOCK_QUANTITY
-------------------------------------------------- ---------- --------------
Laptops
Electronics Gadgets and Devices                        67500              2

Croptops and shirts
Men , Women and Kids Apparels                          45000            456

Television
Home essentials and needs                              75000              5


PRODUCT_NAME
--------------------------------------------------------------------------------
DESCRIPTION                                        PRICE STOCK_QUANTITY
-------------------------------------------------- ---------- --------------
Tiles
Kitchen Needs                                          55000            670
```

Report 2: Customer Value Report
Display:
● Customer Name
● Total Orders
● Total Spending
● Customer Category

```
SQL> SELECT C.FIRST_NAME, COUNT(O.ORDER_ID) AS TOTAL_ORDERS,
  2  SUM(O.TOTAL_AMOUNT) AS TOTAL_SPENDING,
  3  CASE
  4  WHEN SUM(O.TOTAL_AMOUNT)>=100000 THEN 'Premium'
  5  WHEN SUM(O.TOTAL_AMOUNT)>=50000 THEN 'Regular'
  6  ELSE 'Basic'
  7  END AS CUSTOMER_CATEGORY
  8  FROM CUSTOMER C, ORDERS O
  9  WHERE C.CUSTOMER_ID=O.CUSTOMER_ID
 10  GROUP BY C.CUSTOMER_ID, C.FIRST_NAME;

FIRST_NAME           TOTAL_ORDERS TOTAL_SPENDING CUSTOME
-------------------- ------------ -------------- -------
Beema                           2         125000 Premium
Anu                             2          78400 Regular
Reena                           2          67500 Regular
priya                           2         112500 Premium
Arun                            2          48400 Basic
```

Report 3: Sales Performance Report
Display:
● Product-wise sales

```
SQL> SELECT P.PRODUCT_NAME, SUM(OD.QUANTITY) AS TOTAL_SALES
  2  FROM PRODUCT P, ORDER_DETAILS OD
  3  WHERE P.PRODUCT_ID=OD.PRODUCT_ID
  4  GROUP BY P.PRODUCT_NAME;

PRODUCT_NAME
--------------------------------------------------------------------
TOTAL_SALES
-----------
Laptops
          2

Croptops and shirts
          2

Fish tank
          2


PRODUCT_NAME
--------------------------------------------------------------------
TOTAL_SALES
-----------
Unruled notes
          1

Television
          1

Tiles
          1


6 rows selected.
```

- Category-wise revenue

```
SQL> SELECT C.DESCRIPTION AS CATEGORY, SUM(OD.QUANTITY*OD.PRICE) AS REVENUE
  2  FROM CATEGORY C, PRODUCT P, ORDER_DETAILS OD
  3  WHERE C.CATEGORY_ID=P.CATEGORY_ID
  4  AND P.PRODUCT_ID=OD.PRODUCT_ID
  5  GROUP BY C.DESCRIPTION;

CATEGORY                                           REVENUE
-------------------------------------------------- ----------
Kitchen Needs                                       55000
Electronics Gadgets and Devices                    135000
Men , Women and Kids Apparels                       90000
Notebooks,Pens and Papers                            6800
Home essentials and needs                           75000
New Spares for Mobile                                7500

6 rows selected.
```

- Top-selling products

```
SQL> SELECT P.PRODUCT_NAME, SUM(OD.QUANTITY) AS TOTAL_SALES
  2  FROM PRODUCT P, ORDER_DETAILS OD
  3  WHERE P.PRODUCT_ID=OD.PRODUCT_ID
  4  GROUP BY P.PRODUCT_NAME
  5  HAVING SUM(OD.QUANTITY)=(SELECT MAX(SUM(QUANTITY))
  6  FROM ORDER_DETAILS
  7  GROUP BY PRODUCT_ID);

PRODUCT_NAME
--------------------------------------------------------------------------------
TOTAL_SALES
-----------
Laptops
          2

Croptops and shirts
          2

Fish tank
          2
```

Report 4: Business Decision Report

Identify:

● High-performing products

```
SQL> SELECT P.PRODUCT_NAME, SUM(OD.QUANTITY) AS TOTAL_SALES
  2  FROM PRODUCT P, ORDER_DETAILS OD
  3  WHERE P.PRODUCT_ID=OD.PRODUCT_ID
  4  GROUP BY P.PRODUCT_NAME
  5  HAVING SUM(OD.QUANTITY)=(SELECT MAX(SUM(QUANTITY))
  6  FROM ORDER_DETAILS GROUP BY PRODUCT_ID);

PRODUCT_NAME
--------------------------------------------------------------------------------
TOTAL_SALES
-----------
Laptops
          2

Croptops and shirts
          2

Fish tank
          2
```

● Low-performing products

```
SQL> SELECT P.PRODUCT_NAME, SUM(OD.QUANTITY) AS TOTAL_SALES
  2  FROM PRODUCT P, ORDER_DETAILS OD
  3  WHERE P.PRODUCT_ID=OD.PRODUCT_ID
  4  GROUP BY P.PRODUCT_NAME
  5  HAVING SUM(OD.QUANTITY)=(SELECT MIN(SUM(QUANTITY))
  6  FROM ORDER_DETAILS GROUP BY PRODUCT_ID);

PRODUCT_NAME
--------------------------------------------------------------------------------
TOTAL_SALES
-----------
Unruled notes
          1

Television
          1

Tiles
          1
```

● High-value customers

```
SQL> SELECT CUSTOMER_ID, SUM(TOTAL_AMOUNT) AS TOTAL_SPENDING
  2  FROM ORDERS
  3  GROUP BY CUSTOMER_ID
  4  HAVING SUM(TOTAL_AMOUNT)>(SELECT AVG(SUM(TOTAL_AMOUNT))
  5  FROM ORDERS GROUP BY CUSTOMER_ID);

CUSTOMER_ID          TOTAL_SPENDING
-------------------- --------------
C-1001                       125000
C-1002                       112500
```

- Revenue trends

```
SQL> SELECT TO_CHAR(ORDER_DATE,'MONTH') AS MONTH,
  2  SUM(TOTAL_AMOUNT) AS REVENUE
  3  FROM ORDERS
  4  GROUP BY TO_CHAR(ORDER_DATE,'MONTH');

MONTH                                     REVENUE
----------------------------------- ----------
SEPTEMBER                                 431800
```